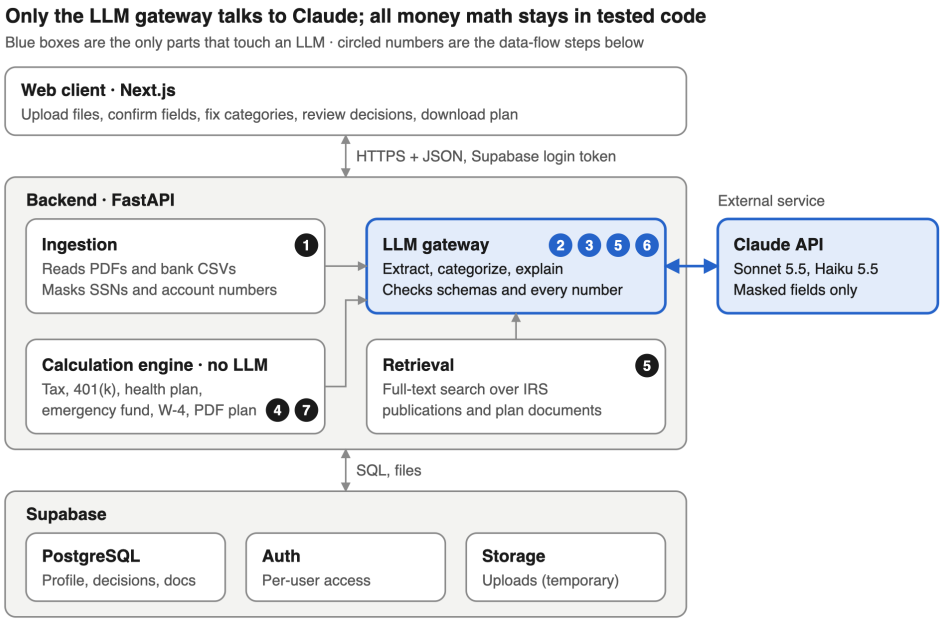
First Paycheck Financial Setup: Technical Spec

First Paycheck is a web app that guides a new salaried employee through their first-month money decisions, using their own pay stub, bank history and benefits documents. All financial math runs in deterministic, tested code. The LLM only extracts, categorizes and explains.

1. Architecture

A Next.js client, a FastAPI backend and Supabase (database, login, file storage). **Only the LLM gateway calls Claude**; the calculation engine does all money math with no LLM.



Data flow (numbers match the diagram):

1. **Upload:** The user uploads a pay stub or offer letter, benefits summary and bank CSV; ingestion masks SSNs and account numbers.
2. **Extract (LLM):** Claude returns typed JSON fields, Pydantic checks them, and the user confirms each one.
3. **Categorize (LLM):** Claude labels transactions in batches with a confidence score; the user can fix any label.
4. **Calculate (no LLM):** The calculators run on the confirmed profile and the results are saved as decisions.
5. **Explain (LLM + retrieval):** From calculator numbers and IRS/plan passages, Claude explains each decision.
6. **Check:** Unsourced numbers or citations are rejected; one retry, then the plain calculator result.
7. **Export:** Decisions become a one-page PDF plan; raw uploads are deleted after confirmation.

2. API Design

REST and JSON (multipart for uploads). Every route needs a Supabase login token and can only reach the user's own data.

Endpoint	Purpose: input → output	LLM
`POST /docs`	Upload: PDF + type (pay stub, offer letter, benefits) → `document_id`	No
`POST /docs/{id}/extract`	Paycheck decoder: `document_id` → fields, confidence, per-line notes	Sonnet 5.5
`PUT /profile`, `GET /profile`	Confirmed salary, benefits, plan options, goals → saved profile	No
`POST /transactions/import`	Spending map: CSV → categorized transactions, confidence	Haiku 5.5
`PATCH /transactions/{id}`	New category → updated transaction	No
`GET /spending/summary`	Date range → totals by category, monthly essentials	No
`POST /calc/emergency-fund`	Essentials, savings, rates → 3–6 month target, months to goal	No

`POST /calc/401k-match`	Salary, match formula → % for full match, $ gained or missed	No
`POST /calc/health-plans`	Premiums, deductibles, HSA, usage → cost per plan, break-even	No
`POST /calc/w4`	Filing status, pay, other jobs, dependents → W-4 values, refund or due	No
`POST /explain`	`decision_id` → concept, numbers, what-ifs, citations	Sonnet 5.5 + RAG
`GET /plan`, `GET /plan/export`	→ saved decisions with reasoning; one-page PDF plan	No

LLM calls: Prompts are versioned in the repo, every output is checked against a Pydantic schema, and each call gets only the masked fields it needs. Failures fall back to the calculator result or manual entry. Stretch: Plaid sandbox sync adds `POST /bank/link` to the same pipeline.

3. Technical Stack

- **Frontend:** TypeScript, Next.js (React), Tailwind CSS, Recharts. **Backend:** Python 3.12, FastAPI, Pydantic v2, versioned IRS Pub 15-T tables
- **Parsing, LLM, retrieval:** pdfplumber, pandas; Anthropic SDK (Sonnet 5.5, Haiku 5.5); Postgres full-text search
- **Data, auth, storage:** Supabase (PostgreSQL, Auth, Storage), SQLAlchemy + Alembic. **PDF export:** WeasyPrint
- **Testing:** pytest, Hypothesis, Vitest, Playwright, custom eval runner
- **CI/CD, hosting:** GitHub Actions (lint, tests, evals; merges blocked on regressions), Vercel, Render

4. Business Context Summary

Problem. In their first month, a new employee must fill out a W-4, pick a health plan, set a 401(k) contribution and work out what they can safely spend, usually with no training. Budgeting apps ignore benefits and taxes, advisors cost too much on a first salary, and articles don't use the reader's own numbers.

Target users. Early-career employees in their first 30 days at a first salaried job with benefits, who return at open enrollment, tax season and raises. **Business model:** free for individuals, or sold to employers as a financial-wellness benefit for new hires.

Success metric	Target	How we measure
Extraction accuracy	≥ 95% of fields	30 labeled synthetic stubs, offers, benefit summaries (CI)
Transaction categorization accuracy	≥ 90%	500 labeled transactions (CI)
Calculator correctness	100%	Tests vs. hand-worked IRS and plan examples
Explanations that cite a source	100%	Automated check on eval outputs
Setup finished in 2 sittings or fewer	≥ 80% of users	User test with 5–8 people
Understanding gain	+2 points avg.	5-question quiz, before vs. after setup

5. Agent Readiness

We use Claude Code and Cursor to write, test and review code, so the repo gives agents the same context a teammate would get.

- **Context files:** a root `CLAUDE.md` (mirrored in `.cursor/rules`) states the product, the user, the rule that money math lives only in `api/calc/` with no LLM, the repo map and the run, test and lint commands; `web/`, `api/` and `evals/` each add a short `CLAUDE.md`.
- **Specs before code:** each feature starts as a short spec in `docs/specs/` with inputs, outputs and hand-worked examples; agents build code and tests from it, and the spec changes first when requirements do.
- **Fast, safe feedback:** `make setup`, `make test` and `make eval` run locally in under a minute; synthetic fixtures and a `.env.example` mean agents never see real data or keys; TypeScript types are generated from the Pydantic schemas so client and server can't drift.
- **Guardrails and log:** pre-commit (ruff, mypy, eslint), CI that blocks merges on test or eval regressions, and AI review on every PR plus a human approval; `docs/ai-log.md` records key prompts, failures and fixes for our reflections.